Agentic Architect: An Agentic AI Framework for Architecture Design Exploration and Optimization

Vasilis Kypriotis
Carnegie Mellon University
Pittsburgh, PA, USA

Alexander Blasberg
Carnegie Mellon University
Pittsburgh, PA, USA

Alexandros Moiras
Carnegie Mellon University
Pittsburgh, PA, USA

Dimitrios Skarlatos
Carnegie Mellon University
Pittsburgh, PA, USA

arXiv:2604.25083v2 [cs.AI] 12 Oct 2026

Abstract

Computer Architecture research has traditionally required the exploration of vast, highly complex design spaces through manual human effort. Large Language Models (LLMs) offer an opportunity to accelerate this process by enabling fast and automated implementation and testing of different architectural mechanisms and their combinations.

We introduce Agentic Architect, a framework for agentic CPU microarchitectural design-space exploration and optimization. Agentic Architect combines LLM-driven code evolution with cycle-accurate simulation and hardware cost modeling to explore performance, area, and power tradeoffs. Starting from a seed design, the LLM is prompted to explore the design space by implementing changes, guided by prior evaluation feedback. Each new design is compiled, simulated on training workloads, evaluated against the optimization objective, and constrained by area and power budgets. Its score and diagnostics update the candidate population and guide subsequent iterations.

We evaluate Agentic Architect across prefetching, cache replacement, and branch prediction. On 213 traces spanning five benchmark suites, its evolved prefetchers lead the evaluated L1D and L2 designs, delivering up to 6.7% and 2.6% additional speedup, respectively, over the strongest state-of-the-art design, while the best evolved cache replacement design improves over Mockingjay by up to 1.7%. Moreover, across 191 branch-prediction traces, the evolved branch predictor achieves 97.62% of TAGE-SC-L's geometric mean IPC at 10.58% of its area. Our analysis highlights the importance of seed selection, hardware constraints, and training workloads in shaping search outcomes. Overall, Agentic Architect establishes the first end-to-end framework for agentic CPU microarchitectural design optimization, opening a new direction for architecture research. The framework will be open sourced upon publication.

CCS Concepts: • **Computer systems organization** → **Architectures**; • **Computing methodologies** → **Search methodologies**.

Keywords: Agentic AI, LLM, evolutionary search, cache replacement, prefetching, branch prediction, microarchitecture

1 Introduction

With the slowdown of Moore's Law and the rising cost of larger on-chip structures, improving processor performance and efficiency increasingly depends on architectural design-space exploration. Decisions about mechanisms, interacting signals, control logic, and hardware organization create a vast combinatorial space. Architects traditionally explore this space by proposing a design, implementing it, simulating it, inspecting the results, and iterating. Each design must be evaluated for performance across workloads and respect area, power, and timing constraints. The effort required limits how many implementations architects can examine, leaving potential combinations and refinements unexplored.

These challenges make a case for an agentic approach that automates and accelerates the architectural design loop. Recent agentic AI systems pair LLMs with evolutionary search [6, 14, 30, 33, 39, 42, 49, 55], demonstrating the potential for AI-driven discovery across scientific and engineering domains. Computer architecture is a compelling but demanding target for this approach. Designs are naturally expressed as modular code, cycle-accurate simulators enable consistent evaluation, and the underlying design space is large but well-structured. However, exploring architectural design remains difficult because performance is workload-dependent, hardware constraints limit feasible designs, and performance depends on multiple interacting mechanisms and signals. The central question of this paper is whether LLMs can generate architectural designs competitive with the state-of-the-art (SoTA), and uncover further gains in performance and efficiency.

To explore that, we introduce Agentic Architect, an agentic framework for CPU microarchitectural design-space exploration and optimization. It connects AI-driven code evolution with cycle-accurate simulation and hardware cost modeling to generate, evaluate, and refine candidate designs under a configurable objective. The framework is initialized with a seed design, LLM, evolutionary framework, architectural simulator, hardware model, task-specific prompt, training workloads, and hardware budgets, which together define the search space and evaluation conditions.

The evolutionary loop starts from the seed population. At each iteration, the search selects a design, combines its code with prior feedback, and prompts the LLM to produce a modified candidate design. The design is validated and evaluated for performance, area, and power. Candidates that satisfy the design constraints are scored against the optimization objective. These scores guide population updates and subsequent selection, while diagnostics inform further changes. After the iteration budget is exhausted, the highest-scoring design is evaluated on the full benchmark suite.

The framework's modular interfaces allow LLMs, evolutionary frameworks, simulators, and hardware models to be changed independently. Architects can therefore select tools appropriate to each architectural domain and configure the seed, optimization objective, and hardware constraints. This makes Agentic Architect a co-design framework: the architect establishes the search conditions, while the AI agent explores within them.

We instantiate Agentic Architect in data prefetching at both L1D and L2 caches, cache replacement and branch prediction, and we examine the factors that shape the evolution. First, seed selection presents a tradeoff between refining a strong design and exploring from a simpler seed. Both approaches prove useful in our experiments: the mature VA/AMPM prefetcher [21] provided the foundation for the best evolved L1D design, while a basic stride prefetcher led to the best evolved L2 design. In replacement, even a simple LRU seed evolved into a competitive design.

Hardware budgets are equally important: without area and power constraints, evolution can gain performance simply by expanding hardware state. Being bounded, the evolved designs improve over strong baselines with modest hardware cost, including only 3.7% additional L1D area over MLOP [48] for the evolved L1D prefetcher, and 0.4% additional LLC area over Mockingjay [47] for the evolved replacement design.

Finally, training-set composition is critical for evolving designs that generalize. Therefore, we selected training sets spanning diverse cache-miss or branch-misprediction rates, to cover a wider range of workload behaviour. Comparing speedups across the training traces and held-out traces shows that the evolved designs retain their performance advantages on unseen workloads.

Observing the evolved designs reveals interesting architectural insights. First, evolution tends to combine well-established mechanisms and signals into coordinated designs. Both evolved prefetchers combine several engines with local control or a central cost gate. Across the designs, it combines established signals, such as memory-level parallelism (MLP) and prefetch lateness, with new derived signals, such as cross-instruction reuse and relative memory-bank traffic. This is the behavior we sought to enable: exploring combinations whose manual refinement would require extensive experimentation. Under fixed hardware budgets, the challenge is not simply adding more state, but finding combinations retaining their benefits while sharing limited resources.

Finally, to assess the performance of evolved designs, we evaluate the evolved prefetching and replacement designs across 213 traces spanning SPEC CPU2006 [53], SPEC CPU2017 [54], PARSEC [13], Ligra [51] and Google Workload Traces v2 [2]. We report geometric mean Instructions-Per-Cycle (IPC) speedup on a single core and geometric mean weighted speedup on four cores.

In prefetching, the evolved L1D design achieves speedups of 1.349× over no prefetching on a single core and 1.356× on four cores. These provide an additional 4.9% and 6.7% speedup over MLOP [48], the strongest evaluated published L1D design, and 7.8% and 8.4% over its VA/AMPM seed [21]. At L2, the evolved prefetcher achieves speedups of 1.434× and 1.420×, outperforming all evaluated L2 prefetchers, including SPP+PPF [12], the strongest published design on our workloads, by 2.6% and 2.0%.

In cache replacement, the best evolved design reaches 1.042× and 1.059× speedup over LRU on one and four cores, improving on Mockingjay by 1.4% and 1.7%, respectively. Starting from LRU, evolution discovers an MLP-based cost design that ranks lines by expected miss cycles saved rather than Mockingjay's Estimated Time of Reuse (ETR). Its better balance across workloads, particularly on unseen held-out traces, allows it to outperform Mockingjay overall.

Lastly, we evaluate branch prediction across 191 traces from the five benchmark suites, using IPC per watt as the optimization objective to explore tradeoffs beyond raw performance. The evolved predictor retains 97.62% of TAGE-SC-L's [46] geometric mean IPC, while reducing modeled area and power by 89.42% and 4.62%, respectively. This demonstrates that agentic search can identify efficient designs under objectives that balance performance, area, and power.

In summary, the contributions of this paper are:

- Agentic Architect, the first framework for agentic CPU microarchitectural optimization, built on a modular architecture for evolutionary frameworks, LLMs, cycle-accurate simulators, and hardware modeling.
- A systematic analysis of how seed selection, hardware budgets, optimization objectives, and training workloads shape evolution, including evaluation on held-out workloads to assess generalization.
- An analysis of what evolutionary search can discover: combinations of established mechanisms and signals, and more efficient organizations under objectives beyond raw performance.
- Evolved designs that optimize performance or efficiency across three domains: cache replacement, data prefetching and branch prediction.
- Open-source release: The Agentic Architect framework will be open-sourced following publication.

2 Background

Agentic Architect combines LLM-driven code evolution with cycle-accurate simulation for computer architecture design exploration and optimization. This section discusses the three domains and the evolutionary framework we build on.

2.1 Microarchitecture Design

Cache replacement. LLC replacement decides which lines to evict, how to prioritize incoming lines, and whether to bypass the cache. These decisions depend on expected reuse: LRU tracks recency, DRRIP [23] adapts insertion priorities, and PC-based predictors learn reuse from program context [56, 57]. Hawkeye [22] learns cache-friendly labels from Belady's optimal policy, while Mockingjay [47] predicts reuse distances to rank lines. Miss cost also matters: memory-level parallelism (MLP) overlaps requests, so retaining a line whose miss would stall execution can be more valuable [41].
Hardware Prefetching. Hardware prefetchers issue requests ahead of demand to hide latency and increase MLP. Common designs use prediction engines that capture complementary access patterns: next-line and PC-stride target regular accesses, spatial prefetchers use access maps or footprints [8, 21, 52], delta predictors such as Berti [38] and SPP [28] learn address differences, and MLOP [48] scores offsets at multiple lookahead depths. These mechanisms predict both which data to fetch and how far ahead.

Combining prediction engines requires coordinating request issuance and fill level. Coverage, accuracy, and timeliness capture the resulting tradeoffs, as greater aggressiveness can increase coverage at the cost of bandwidth and cache pollution. Prefetchers therefore use filtering and adaptive feedback to balance these tradeoffs: PPF [12] rejects predictions likely to be unused, DSPatch [11] adjusts coverage and accuracy according to bandwidth, and reinforcement-learning prefetchers such as Pythia [10] learn actions from rewards for usefulness, timeliness, and bandwidth conditions.
Branch Prediction. Branch predictors use history to predict branch outcomes and avoid costly mispredictions. Perceptron-based designs [25–27] combine learned weights from history features, while TAGE [45] selects predictions from tagged tables using different history lengths. TAGE-SC-L [46] adds a statistical corrector and loop predictor, forming a strong baseline for exploring performance and hardware efficiency.

2.2 LLM-Driven Code Evolution

Agentic Architect performs architectural exploration through evolutionary search, using LLMs to generate candidate design mutations. AlphaEvolve [39] demonstrated this approach, while OpenEvolve [49] provides an open-source framework following the same ideas. Its evolutionary loop has four stages: the *LLM Evolver* mutates a selected implementation; the *Evaluator* compiles and benchmarks the candidate; the *Solution Database* stores candidates in islands to maintain diversity; and *Prompt Creation* selects a parent and constructs the next prompt using evaluation feedback. Further works adapt island selection and exploration to search progress [14], or improve efficiency through parent sampling, novelty filtering, and bandit-based LLM selection [30].

Computer architecture suits this approach: designs are modular logic blocks that LLMs can modify, while cycle-accurate simulators and benchmarks provide an established evaluation loop. However, architectural evolution must further consider how mechanisms and signals interact with the core and memory hierarchy, alongside their hardware costs. Agentic Architect addresses these requirements by enforcing modeled area and power budgets and guiding code changes with feedback on architectural behavior, according to the architectural domain. We discuss these challenges through the design and evaluation of Agentic Architect.

3 Agentic Architect's Design

The Agentic Architect framework automates microarchitectural design-space exploration as a constrained optimization process. It generates, evaluates, and refines candidate designs toward an architect-defined objective, such as IPC or performance per watt, while enforcing hardware budgets throughout the search.

Figure 1 illustrates the end-to-end workflow. Evolution is initialized at Step ⓪ by a baseline design called *the seed* and a static prompt, which provides initial guidance for the search. The remaining inputs, marked by gears in the figure, configure the framework: the LLM and evolutionary framework, simulator and hardware model, trace database, optimization objective, and hardware budgets. Together, these configurations remain fixed throughout the evolutionary loop.

At each iteration, design evolution (Step ①) produces candidate designs using the chosen LLM and evolutionary framework. Each generated candidate is passed to the evolutionary agent (Step ②), which checks its validity and hardware feasibility. Accepted candidates are executed on the training traces by the microarchitectural simulator (Step ③), which returns execution metrics to the evaluator (Step ④). The evaluator converts these measurements into a score to guide subsequent iterations. Once the iteration budget is exhausted, the highest-scoring design is evaluated on the held-out traces. The remainder of this section describes the framework by following the evolutionary workflow.

3.1 Initialization and Configuration

Architectural design tasks differ in evaluation tools, objectives, constraints and more. For instance, tasks may use cache misses, prefetch coverage, or branch mispredictions as metrics, permit changes to mechanisms or storage organization, and optimize IPC or performance per watt under different hardware budgets. Agentic Architect separates these choices

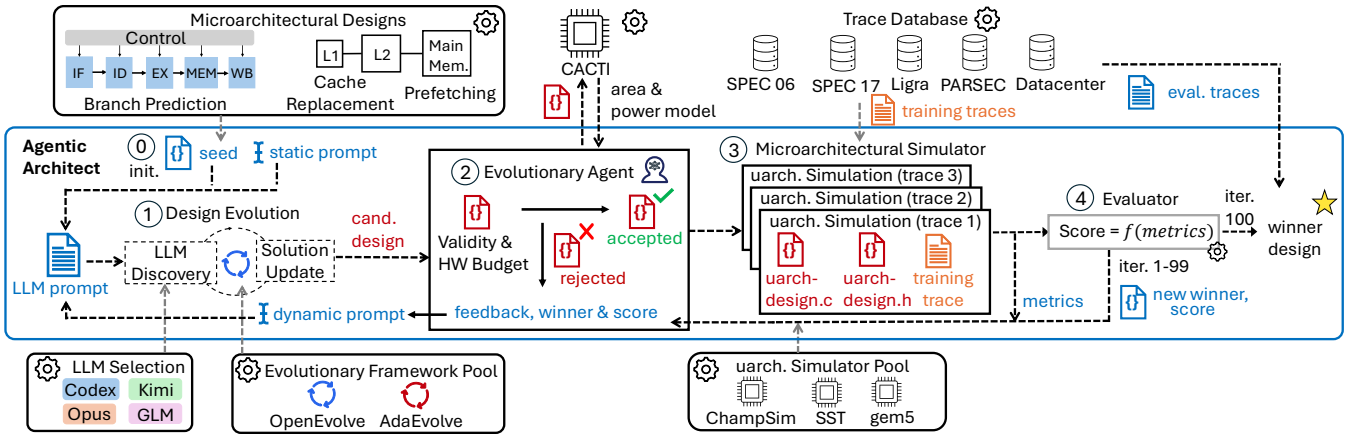


Figure 1. Overview of the Agentic Architect framework. Steps ⓪–④ show initialization, design evolution, candidate validation, microarchitectural simulation, and evaluation. Gear icons mark configurable framework components.

into *evolution initialization*, which defines the starting point for search, and *framework configuration*, which determines how candidates are generated and evaluated.

Initialization (Figure 1, Step ⓪) provides the seed implementation and static domain prompt. The seed defines the starting design, while the prompt provides fixed guidance on mechanisms, architectural signals, or design restrictions.

The framework's configuration inputs, marked by gear icons in Figure 1, include the LLM and evolutionary framework, simulator and hardware model, trace database, scoring function, and hardware budgets. Hardware modeling provides cost estimates and budget checks, simulation returns performance metrics and diagnostics, and the scoring function quantifies evaluation results, reflecting the optimization objective. Training workloads provide feedback during evolution, while held-out workloads evaluate the final design.

Defined component interfaces make Agentic Architect modular, allowing different search methods, LLMs, simulators, and hardware models to be integrated into the same workflow. Beyond incorporating new tools and models, this modularity supports different architectural domains that require their own evaluation tools and objectives, without rebuilding the optimization loop. Once initialized and configured, these choices remain fixed throughout a run, so all candidates are evaluated under the same conditions.

3.2 Design Evolution

Design evolution (Figure 1, Step ①) is performed by the evolutionary framework and LLM. At each iteration, the framework selects a parent from the design population and receives a dynamic prompt composed of its design and prior feedback. The LLM then produces an evolved design. The search remains bounded by the architect-defined design directives and requirements. The candidate is then passed to the evolutionary agent (Figure 1, Step ②) for validation and evaluation.

3.3 Agentic Architect's Evolutionary Agent

The evolutionary agent (Figure 1, Step ②) sits between design evolution and microarchitectural simulation, and its role is to turn the selected evolutionary framework into a constrained architectural design search. It coordinates candidate generation, validation, simulation, and scoring, returning both scores and failures to the LLM for subsequent evolutions. Because evolved designs may be invalid, hardware-infeasible, or prohibitively expensive to evaluate, the agent enforces two design principles.

Machine-checkable candidates. Generated designs follow a predefined output contract that enables automatic validation and prevents non-executable candidates from consuming simulation time.

Hardware feasibility during search. Area and power are enforced as search constraints. As shown in Figure 1, the hardware model evaluates each candidate before simulation and rejects overprovisioned designs. Later, execution activity enables dynamic-power validation. This both prevents evolution from buying performance with additional hardware and avoids wasting compute on already-infeasible candidates.

Accepted candidates proceed to the simulator (Figure 1, Step ③), while rejected ones return negative scores and failure diagnostics, steering evolution away from invalid designs. The agent spans architectural domains, turning generic evolutionary search into architectural exploration through validity checks, hardware constraints, and simulator feedback.

3.4 Integration with Microarchitectural Simulators

The simulator (Figure 1, Step ③) serves as an interchangeable execution backend behind a fixed interface. This interface separates the evolutionary search from simulator-specific details and has two roles.

Design interface. For each architectural domain, the simulator exposes the points where an evolved design may act

and the signals it may observe. A replacement design, for example, selects victims and updates replacement state, while a prefetcher observes cache accesses and proposes requests. Evolution may change the mechanism within this boundary, but not the surrounding microarchitecture. The interface also exposes hardware activity needed for power modeling.
Measurement interface. The simulator executes accepted candidates on the training traces and reports performance measurements and domain-specific statistics, such as IPC, cache misses, or branch mispredictions to the evaluator.

This abstraction makes Agentic Architect portable across simulators and architectural components: a new backend only needs to expose the corresponding design and measurement interfaces, without changing the evolutionary search.

3.5 Agentic Architect's Evaluator

The evaluator (Figure 1, Step ④) translates simulator measurements into the scalar fitness that drives evolution. Its configurable scoring function allows the architect to define the optimization objective. For example, it can optimize raw IPC, performance per watt, or domain-specific metrics.

Importantly, scoring is separated from diagnostic feedback. Metrics such as cache misses, and branch mispredictions may be reported without affecting overall ranking, helping explain design improvements or regressions. Scores and diagnostics guide subsequent evolutions, and the highest-scoring design is evaluated in the end on held-out traces.

3.6 Prompt Design

The prompt is the harness through which Agentic Architect steers the LLM during evolution. It combines a static prompt provided at initialization with a dynamic prompt constructed from feedback during the search.
Static prompt. As described earlier, the static prompt is written once by the architect and remains fixed throughout evolution. It communicates the design goal, the seed, and any guidance the architect wants the LLM to use.
Dynamic prompt. The dynamic prompt provides live feedback from the evolutionary search back to the LLM. For each iteration, it includes the selected parent design together with its score, diagnostic metrics, and feedback from prior evaluations. For example, the LLM can learn that a candidate improved IPC but increased misses, or that a previous iteration violated a hardware constraint.

3.7 The Role of the Architect

Several inputs and configurations remain explicit architectural decisions: the seed that initializes search, the objective it optimizes, the constraints candidates must satisfy, the workloads used for training, and the guidance provided through the prompt. Agentic Architect deliberately leaves these choices open because their effectiveness depends on the architectural domain and optimization goal.

Table 1. Configuration of evaluated replacement designs.

DRRIP [23]	2-bit RRPV/line, 10-bit PSEL, 5-bit BRRIP counter	**8.0 KiB**
Hawkeye [22]	3-bit RRPV/line, 2×2K SHCT, 2800-entry sampler, 32 × 128 OPTgen	**30.7 KiB**
Mockingjay [47]	5-bit ETR/line, 2K-entry RDP, 5-way sampled cache	**33.1 KiB**
EVO-LRU	128-set hit count., LLC set state, 32-bank heat, 4K-entry PC cost, 4b line cost	**33.4 KiB**
SHiP [56]	2-bit RRPV/line, 16K-entry SHCT (3-bit), 4K-entry sampler	**34.0 KiB**
EVO-MJ	Mockingjay, 1-bit per-line miss-cost, 8K × 5-bit no-reuse counters	**42.1 KiB**

This division of responsibility makes Agentic Architect a co-design workflow: the architect defines the search conditions and what constitutes progress, while the LLM explores designs within them at a scale impractical for manual iteration. In Section 6 we evaluate the impact of several of these decisions on the resulting designs.

4 Use Cases for Microarchitectural Design Exploration

We instantiate Agentic Architect across three extensively studied microarchitectural domains: cache replacement, data prefetching, and branch prediction. For each use case, we select established designs as seeds and evolve them under specified objectives and hardware budgets. We first introduce the search strategy, then describe the evolved mechanisms and signals, and finally highlight how the resulting designs improve upon their seeds.

4.1 Use Case 1: Evolving Cache Replacement Designs

Cache replacement is our first use case. We seed evolution with Mockingjay [47], a SoTA reuse-distance predictor, producing EVO-MJ, and evaluate final results relative to LRU as a baseline. We also seed evolution from LRU, producing EVO-LRU, to study seed sensitivity (Section 6.3). We also compare against Hawkeye [22], SHiP [56], and DRRIP [23]. Table 1 summarizes these designs, along with the evolved ones, their configurations, and their storage requirements.

4.1.1 EVO-MJ: A cost and stream-aware Mockingjay.

Figure 2 illustrates EVO-MJ, which preserves Mockingjay's reuse-distance predictor (RDP), sampled training, and expected-time-to-reuse (ETR) ranking. It adds a per-line miss-cost bit for resolving victim ties and a per-PC no-reuse counter that supplies an ETR when the RDP has no prediction. Our analysis attributes the performance gain primarily to the latter, which improves line retention and LLC load hits; cost awareness only refines otherwise-equivalent victim choices.
LLC access and no-reuse confidence. As shown by the lower path in Figure 2, LLC accesses still use Mockingjay's RDP and ETR ranking. EVO-MJ adds a 5-bit per-PC no-reuse

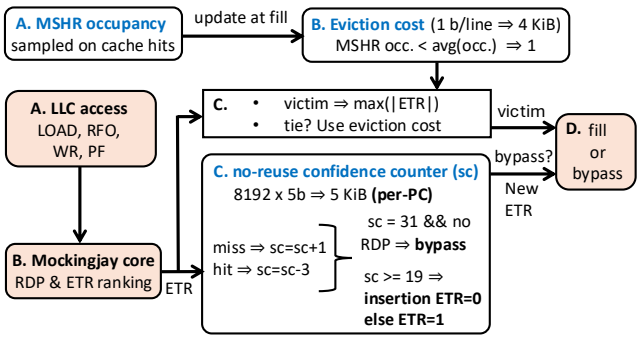


Figure 2. EVO-MJ architecture. Orange components are inherited from Mockingjay. The remaining mechanisms are introduced by evolution.

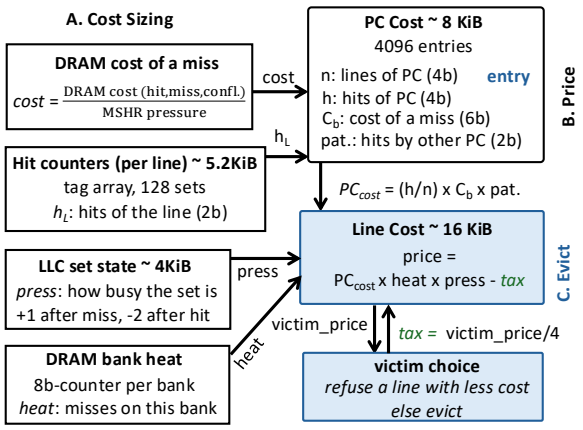


Figure 3. The architecture of EVO-LRU, the evolved cache replacement design from LRU.

counter, *sc*, for LOADs and Requests For Ownership (RFOs): misses increment *sc*, while hits decrement it by three. When the RDP has no prediction, $sc \geq 19$ gives ETR = 0; otherwise ETR = 1. Prefetch and WRITE handling remain unchanged. This path drives most of EVO-MJ's gain by distinguishing likely streaming from other unknown PCs, improving retention and LLC load hits.

Victim selection. As shown by the upper path in Figure 2, EVO-MJ adds a one-bit miss-cost signal per LLC line. At fill, it compares sampled MSHR occupancy with its running average: low occupancy indicates less MLP and thus a more costly future miss, so the line is marked expensive. Mockingjay's |ETR| ranking remains unchanged; EVO-MJ uses this bit only on ties, evicting a cheap line when possible. This preserves lines whose eviction would cause more costly misses without overriding Mockingjay's reuse prediction.

Fill or bypass. EVO-MJ adds one bypass case to Mockingjay: a LOAD or RFO with no RDP prediction is bypassed when its no-reuse counter saturates at $sc = 31$, avoiding cache pollution from accesses with strong no-reuse evidence.

4.1.2 EVO-LRU: Evolved cache replacement from LRU.

EVO-LRU replaces LRU's recency ordering with an estimate of the cycles saved by retaining each line. Like Mockingjay, its goal is to rank each LLC line by a trained value. But instead of using Mockingjay's ETR, EVO-LRU ranks them by the expected miss cycles saved if they are kept. Figure 3 shows how evolution constructs this signal: it first estimates a cost per PC, refines it into a per-line price, and then uses that price for admission and victim selection. In Section 6 we will see how these two signals compare.

PC-Cost sizing. EVO-LRU learns a PC-level cost from miss cost and reuse. For each miss, DRAM row state provides a base cost, which is scaled by MSHR occupancy to estimate how much of that cost is exposed after accounting for MLP. In parallel, a sampled cache tracks a subset of LLC lines and attributes hits to their installing PC, estimating the reuse earned by that PC's lines. The PC table in Figure 3 combines these signals with an evolved cross-PC reuse signal, which captures reuse by PCs other than the installer, to estimate the miss cycles saved by retaining a line.

LLC-Line price. EVO-LRU then refines this PC-level cost using the two signals shown in Figure 3: LLC-set pressure and DRAM-bank heat. Set pressure captures competition for cache capacity, while bank heat is a measure of relative memory-bank traffic, increasing the value of lines mapped to banks receiving a larger share of recent misses. Their combination produces the price stored with each LLC line.

Victim Eviction. EVO-LRU ranks all 16 ways by this price and selects the minimum. An arriving line that cannot beat the victim is bypassed. Otherwise it replaces the victim and pays a fraction of its price as a replacement tax.

Thus, from an LRU seed, evolution independently converges toward a cycle-cost-based replacement design [24, 34, 41], ranking LLC lines by exposed miss cost rather than reuse distance. It further strengthens this MLP-aware ranking with two new derived signals: cross-PC reuse and relative memory-bank traffic.

4.2 Use Case 2: Evolving Prefetching

Data prefetching is our second use case. We seed evolution with VA/AMPM Lite [21], a compact prefetcher in the L1D cache, and report final performance relative to a no-prefetch baseline. We also seed with PC-Stride at the L2 cache, to study seed sensitivity (Section 6.3).

We compare with the following L1D prefetchers: IPCP [40], Berti [38], VA/AMPM [21] and MLOP [48], and the L2 prefetchers: Bingo [8], SPP [28], Pythia [10], and SPP+PPF [12, 28]. Table 2 lists their configuration and storage needs.

4.2.1 EVO-AMPM: The evolved L1D Prefetcher.

EVO-AMPM evolves AMPM from a single bitmap-based predictor into a coordinated multi-engine prefetcher. As Figure 4 shows, each cache access updates shared per-page state, five

Table 2. Configuration of evaluated prefetchers.

L1D Prefetchers		
AMPM [21]	128-entry region table, 64-block access/prefetch maps, degree 2	**3.0 KiB**
Berti [38]	152-entry LT, 8 × 16 history, 64-entry BT (×16 deltas)	**5.7 KiB**
MLOP [48]	128-entry AMT, 500-update, 16-degree	**6.5 KiB**
EVO-AMPM	128-entry region, 256 IP, 256 pattern, 128 NDT, 16 offsets	**8.3 KiB**
IPCP [40]	1K IP table, 4K DPT, 16-entry GHB	**15.1 KiB**
L2 Prefetchers		
SPP [28]	256-entry ST, 512 × 4-way PT, 1K filter, 8-entry GHR	**6.8 KiB**
Pythia [10]	2 features, 3 tilings, 128 tiles, 15 actions; 64-entry ST, 256-entry PT	**26.3 KiB**
Bingo [8]	2 KiB region, 64-entry FT, 128AT, 4K 16-way PHT, 128-entry streamer	**34.3 KiB**
EVO-Stride	1K PC-stride, 256 page-stream, 256 AT, 1K PHT, 1K cost table, 1K issued, 512 shadow	**42.3 KiB**
SPP+PPF [12]	SPP w/ 2K × 4-way PT, 9-feature perceptron (22.7K weights), 2×1K filters	**71.4 KiB**

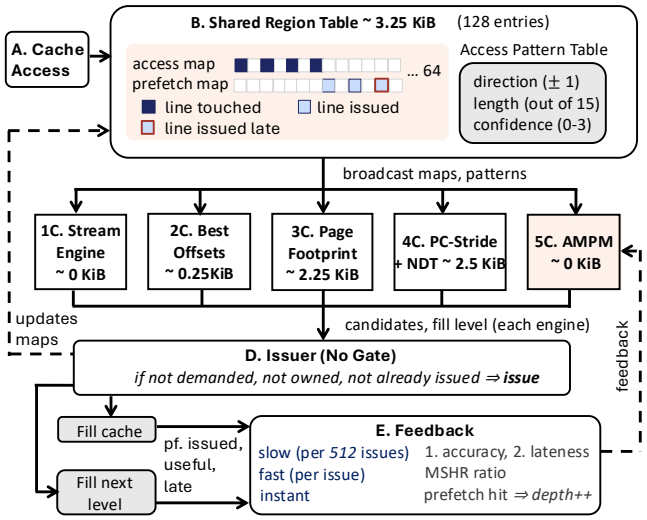


Figure 4. The architecture of EVO-AMPM. The AMPM engine and bitmaps (marked with orange) are inherited from the seed. All other components are introduced by evolution.

complementary engines generate candidates, a common issuer removes redundant requests, and global feedback adapts their aggressiveness and depth. Thus, the gain does not come from a new prediction signal, but from combining existing predictors while coordinating their state and actions.

Shared Region Table and engines. EVO-AMPM retains AMPM's access and prefetch bitmaps and turns them into shared state for all five engines. Evolution augments this state with compact stream and footprint information, allowing Stream, BOP [37], Footprint [52], PC-Stride+NDT [7, 18, 50], and AMPM [21] to target different access patterns from the

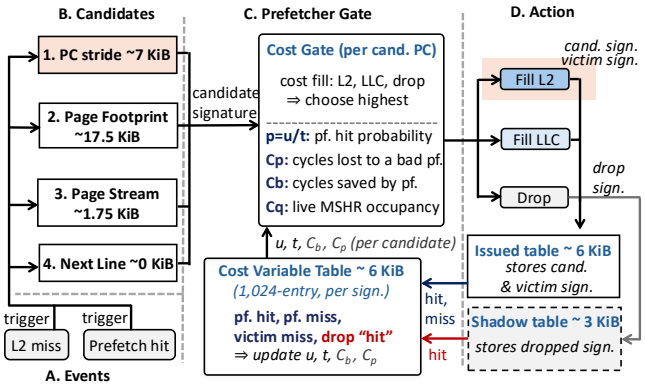


Figure 5. The architecture of EVO-Stride. The PC-Stride and L2 fill only (marked with orange) are properties of the seed. The rest of the design is invented by the evolution.

same page history. The shared bitmaps also expose whether an issued prefetch was useful or late. Compared with the AMPM seed, this broadens prediction coverage while allowing multiple engines to reuse the same per-page history.

Global issuer. As shown next in Figure 4, candidates from all engines meet at a thin global issuer. It only removes candidates targeting the current demand line, duplicate candidates, and already-issued requests. EVO-AMPM therefore preserves the specialization of each engine while preventing their combination from generating redundant traffic.

Feedback Module. Finally, issued prefetches feed accuracy, lateness, MSHR pressure, and prefetch-hit information back to the engines. Evolution uses these signals to adapt active engines and prefetch depth. EVO-AMPM therefore improves over AMPM by expanding the set of recognizable access patterns while dynamically controlling the extra traffic that this broader coverage creates. Total storage is **8.3 KiB** (Table 2).

4.2.2 EVO-Stride: The evolved L2 Prefetcher.

EVO-Stride evolves PC-Stride into a multi-engine prefetcher, combining predictors similar to EVO-AMPM, but replacing global control with a per-candidate cost gate. As Figure 5 shows, four generators propose candidates, which are priced in cycles and independently filled in L2, LLC, or dropped.

Prefetch Engines. The retained PC-Stride engine is complemented by three more prefetching engines: Page Footprint, Page Stream, and Next Line, broadening coverage beyond repeated per-PC strides (Figure 5, Step B). The engines produce candidate signatures that identify both their source and access pattern, allowing later feedback to distinguish predictions from different behaviors.

Cost Gate. The main evolved mechanism is the gate (Figure 5, Step C). For each candidate, it learns the benefit of a successful prefetch, the cost of a harmful one, its hit probability, and current MSHR pressure. These signals are expressed in cycles and used to score three actions: fill L2, fill LLC,

Table 3. Configuration of evaluated branch predictors.

Hashed Perceptron [25]	64K Weight Tables, History Registers	**64 KiB**
Gshare [36]	4K Direction Table, 14-bit GHR	**4 KiB**
Bimodal	4K Direction Table	**4 KiB**
TAGE-SC-L 8/64 KiB [46]	6.9K/56K TAGE, 1K/6.4K SC, <0.1K/0.2K LP, −/1K Histories	**8/64 KiB**
MPP 8/64 KiB [26]	7.97K/58.65K Weight Tables, 0.27K/1.19K Histories/State, −/4.4K Branch Filter	**8/64 KiB**
EVO-TAGE	8×1K 7-bit weight tables, 160 B local histories	**7.17 KiB**

or drop. The highest-valued action is selected. Unlike EVO-AMPM's global feedback, EVO-Stride applies its learned cost model at candidate granularity.

Action and feedback. Issued and dropped candidates are tracked (Figure 5, Step D) so later prefetch hits, misses, victim misses, and demands for dropped lines can update the gate's cost estimates. This feedback lets EVO-Stride issue more aggressively when the predicted cycle benefit outweighs the cost, rather than maximizing prefetch accuracy alone. The resulting gain comes largely from L2 residency: extra prefetches keep working-set lines in the L2 long enough to serve repeated demand accesses. Total storage is **42.3 KiB**.

4.3 Use Case 3: Evolving Branch Prediction

Our final use case is branch prediction, where we optimize IPC per watt, using whole-processor power to account for the pipeline effects of mispredictions. Evolution starts from 64 KiB TAGE-SC-L [46] and produces EVO-TAGE. We compare against Bimodal, 8/64 KiB TAGE-SC-L, 8/64 KiB MPP [26], Gshare [36], and Hashed Perceptron [25]. A summary of all evaluated designs is presented in Table 3.

4.4 EVO-TAGE: The evolved branch predictor

EVO-TAGE deviates form TAGE-SC-L [46], replacing its tagged prediction tables with eight, area-efficient tables of 1024 signed 7-bit weights (Figure 6). For each branch, it reads one weight from each table and combines them into a sum whose sign determines whether to predict taken or not-taken.

Selecting the weights. The eight tables differ in the information used to select a weight. One uses only the current branch's address, providing a bias toward its usual outcome. All seven remaining tables use the address and global history: the taken/not-taken outcomes of recently executed branches, with a different history length (L_{T_i}) for each table. In addition, their hashes directly incorporate their index, multiplied by an odd integer salt S_2, a feature that is not present in MPP [26]. Of these seven tables, one uses no additional history, three also include a 25-bit summary of recent branch addresses (path history), and three also include the current branch's

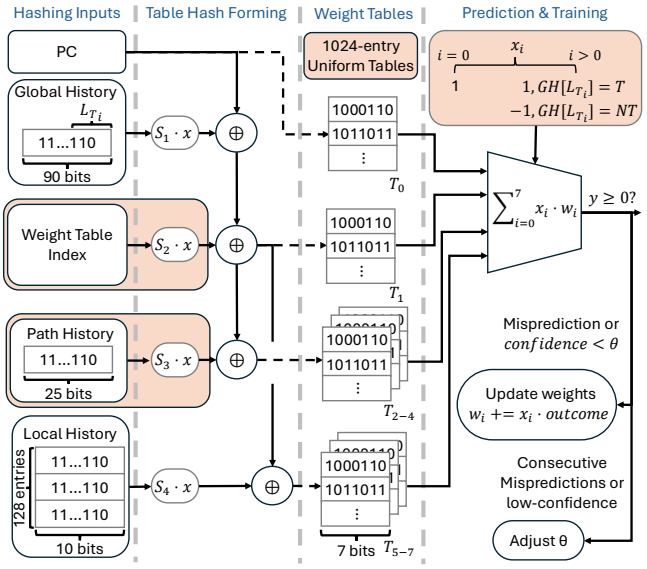


Figure 6. The architecture of EVO-TAGE. The design is closer to MPP [26] with key differences being highlighted in orange.

last ten outcomes (local history). Signal mixing is performed using XOR, which is also applied between individual signals and right-shifted versions of themselves. Examples of such signals are the PC and each table's final hash.

Prediction. The weighted sum from the tables determines the prediction: its sign indicates the branch outcome, and its magnitude indicates confidence. The address-only table contributes its weight directly. Every other table adds or subtracts its weight according to a selected global-history bit, adding for taken and subtracting for not-taken.

Training. The predictor updates the selected weights after a misprediction or when confidence falls below a threshold. The bias increases by one for a taken outcome and decreases by one for not-taken. Each remaining weight increases by one when the actual outcome matches its own specific global history bit, and decreases by one otherwise. These updates move the sum towards the correct outcome, following the principle used in perceptron prediction [27]. The threshold is adjusted during execution aiming to balance update events caused by mispredictions and low-confidence predictions, as in MPP [26] and O-GEHL [44].

Unlike MPP, EVO-TAGE uses uniformly sized tables, includes global history when indexing all history-based tables, and uses global history to decide whether to add or subtract their weights. For path history, it replaces the 16-bit value array used in MPP with a 25-bit rolling hash, which is also updated by non-conditional branches save for direct jumps. Moreover, it ignores many of MPP's independently specified features such as IMLI and MODPATH. The complete predictor requires **7.17 KiB** of state.

Table 4. Simulated ChampSim configuration.

Core	**Cache Replacement & Data Prefetching:** 1–4 cores, 4-wide OoO @ 4 GHz, 256-entry ROB, 64/42-entry LQ/SQ. **Branch Prediction:** 1 core, 6-wide OoO @ 4 GHz, 352-entry ROB, 128/72-entry LQ/SQ.
L1/L2 Cache	Private, 32 KiB (4-way) / 256 KiB (8-way), 64 B line, LRU, 16/32 MSHRs, 1-cycle / 10-cycle round-trip latency.
LLC	2 MiB/core, 16-way, 64 B line, RQ/WQ/PQ/MSHR = 32/32/32/64 (1C), 128/128/128/256 (4C), 24-cycle round-trip latency.
Main Mem.	**1C**: 1 channel, 1 rank/channel; **4C**: 2 channels, 2 ranks/channel; 32 banks/rank, 3200 MT/s, 64-bit data bus per channel, t_{RCD}, t_{RP}, t_{CAS} = 15 ns, t_{RAS} = 32.5 ns.

Table 5. Evaluated Workload Suites.

Suite	#Workloads		#Traces		Example Workloads
	Repl/Pf	Br	Repl/Pf	Br	
SPEC06	15	10	40	22	gcc, mcf, bwaves, …
SPEC17	12	8	39	20	gcc, mcf, fotonik3d, …
PARSEC	5	7	17	25	canneal, raytrace, …
Ligra	13	13	72	80	BFS, BellmanFord, …
Google	12	12	45	44	arizona, delta, tahoe, …
Total	57	50	213	191	

4.5 What does the evolution discover?

Across these five designs, evolution primarily discovers new mechanism combinations and signals rather than entirely novel designs. Both prefetchers combine complementary engines, while EVO-TAGE mixes well-studied branch history signals. Such combinations are non-trivial because mechanisms compete for shared state and bandwidth, and their effects are not additive: improving one can reduce another's benefit. Evolution also introduces new signals that materially improve decisions, including cross-PC reuse and relative memory-bank traffic in EVO-LRU. Thus, the novelty lies in discovering how mechanisms and signals interact under hardware constraints, rather than merely tuning parameters or adding state.

5 Methodology & Experimental Setup

We evaluate all three use cases using the ChampSim cycle-accurate simulator [16], configured as summarized in Table 4. The memory hierarchy is kept identical across all experiments, while ROB capacity and core width follow prior evaluation setups: a 256-entry ROB, four-wide core for replacement and prefetching [1, 3, 10, 47], and a 352-entry ROB, six-wide core for branch prediction [25, 58].

5.1 Benchmark Traces

We evaluate replacement and prefetching on 213 traces from SPEC CPU2006, SPEC CPU2017, PARSEC, Ligra [13, 51, 53, 54], and Google Workload Traces v2 [2]. For replacement and prefetching, all traces have LLC MPKI ≥ 3 on a single-core, no-prefetching, LRU baseline. Branch prediction uses 191 traces, each with branch MPKI ≥ 3. Trace details are summarized in Table 5.

Single-core experiments use 200M warmup and 1B measured instructions, while four-core experiments use 50M warmup and 200M measured instructions per core. Four-core mixes are either homogeneous, with four copies of the same trace, or heterogeneous, with four distinct traces randomly selected from the non-Google traces. We evaluate 100 such heterogeneous mixes. We report single-core performance as geometric-mean IPC speedup and four-core performance as geometric-mean weighted speedup across mixes. Traces that reach their end are replayed from the beginning [10, 47]. We evaluate branch prediction single-core only, since its effects are primarily within the core [25, 45].

5.2 Evolution Setup

We use OpenEvolve [49] as the evolutionary backend. Each run evaluates 100 candidates over 100 iterations. To limit search cost, evolution uses only the single-core configuration with 50M warmup and 200M measured instructions on random subsets of 58 and 64 traces drawn from SPEC06, SPEC17, PARSEC, and Ligra, chosen to span a broad range of LLC MPKI and branch MPKI. Google traces are excluded entirely from evolution, providing an unseen workload set for evaluating generalization. All reported results use the full benchmark sets and the evaluation methodology above.

5.3 Hardware Cost Model

Each candidate declares the capacity, geometry, and organization of its stateful structures, which we model with CACTI 7.0 [9] for area, per-access read/write energy, and leakage power. During simulation, the design reports per-structure read and write counts. Multiplying these counts by their access energies and dividing by simulated execution cycles gives the mechanism's average dynamic power. Adding leakage gives its total power. We follow the same methodology to model the processor's caches and TLB.

For the rest of the processor, McPAT [32] estimates peak dynamic power, scaled by IPC relative to maximum retire width to approximate average activity, with leakage added separately. Together, these estimates capture both the mechanism's direct hardware cost and its effect on processor activity. CACTI and McPAT use a 65 nm technology node.

Unless stated otherwise, area and power budgets are relative to the seed. Our main experiments reject candidates exceeding 30% overhead in either metric.

5.4 LLM, Prompts and Scoring

We use Claude Opus 5 [5] as the LLM in this instance of Agentic Architect. A static system prompt explicitly defines

Prompt template

```
You are an expert computer architect specializing in LLC
replacement policies. Evolve the provided policy [..].

# GOAL
Maximize `combined_score` = geometric mean of IPC across 58
traces, weighting every trace's RELATIVE improvement equally.
LLC miss counts are diagnostic and do NOT affect the score.
Prefer trading raw misses for hits on critical lines. [...]

# BUDGET
Area and power overhead must each stay within 30%.

# TASK
Respond with complete openevolve.{h,cc} in the following format:
// [START OF HEADER FILE: openevolve.h]
...
// [END OF HEADER FILE: openevolve.h]

// [START OF SOURCE FILE: openevolve.cc]
    #include "openevolve.h"
    // ... implementations ...
// [END OF SOURCE FILE: openevolve.cc]

# CACTI CONFIG
At the end of your response, declare each stateful structure your
branch predictor adds (prediction tables, tag tables, history
buffers, loop tables, statistical corrector tables, counters) in
exactly this format:...

# ACCESS COUNTERS
Every structure declared in the CACTI block must maintain read/write
counters, printed in replacement_final_stats() - one line per
structure, name matching the CACTI_STRUCTURE name exactly
, format exactly: ...

# TARGET SYSTEM
- LLC 2 MiB 16-way, 24-cyc, 64 MSHRs | L2 256 KiB | L1D 32 KiB
- 4 GHz, 256-ROB | DRAM 3200 MT/s, 15/45 ns row hit/conflict

# SEARCH STRATEGY
Declare your move on line 1 of the header, then execute it:
 // MOVE: NOVEL   -- progress stalled; invent a mechanism
    matching no published policy, on unused signals.
 // MOVE: IMPROVE -- still gaining; tune tables or graft
    one complementary mechanism.
```

Figure 7. Simplified system prompt for the cache replacement use case, specifying the objective, output contract, target system, available signals, feasibility budgets, and search directive.

```
# GOAL
...Since the branch prediction seed is particularly
strong and achieves very high prediction accuracy, your
focus should be on reducing the design's area and power
while maintaining accurate branch predictions.
```

Figure 8. System prompt goal modification for branch prediction.

the seed policy, optimization objective, and steering directives including the area and power budgets. It also specifies a strict LLM output contract which has to be verified by the evolutionary agent for the design to proceed with the evaluation. Finally, it provides the LLM with the specifications of the evaluated system as well as potential exploration options. A cache replacement evolution example is presented in Figure 7 while the goal shift towards efficiency for the branch prediction use case is presented in Figure 8. During evolution, cache replacement and prefetching candidates are scored by geometric-mean IPC. For branch prediction, we score with IPC per watt, demonstrating that the framework can target a different objective.

6 Evaluation

We evaluate Agentic Architect across cache replacement, data prefetching, and branch prediction, comparing evolved designs with their seeds and SoTA designs. We ask whether evolution can improve mature designs, discover competitive designs from weaker seeds, satisfy hardware budgets, and generalize to unseen workloads and longer simulations.

6.1 Cache Replacement

We seed evolution with Mockingjay, producing EVO-MJ. Figure 9 reports geometric mean speedup over LRU for one and four cores. EVO-LRU is discussed separately in Section 6.3.

EVO-MJ reaches 1.034× and 1.045× speedup over LRU, improving Mockingjay, the strongest baseline, by 0.55% and 0.44%, respectively. Its largest gain over Mockingjay occurs on Ligra. Both designs degrade sharply on the unseen Google traces, reflecting behavior inherited from Mockingjay, although EVO-MJ recovers about 1.2% over its seed.

EVO-MJ wins primarily by correcting accesses for which Mockingjay's RDP has no prediction. Mockingjay assigns such demand lines ETR = 0, allowing streaming accesses to displace reusable data. EVO-MJ's no-reuse counter identifies these streams earlier and bypasses them once confidence saturates, preserving reusable lines. Its miss-cost signal further improves equal-ETR decisions by retaining lines whose future misses would expose more latency. This explains its larger gains when streaming and reusable accesses coexist, as in Ligra, while leaving little room to help workloads with almost no reusable LLC data.

6.2 Data Prefetching

For prefetching, we seed evolution with AMPM at L1D, producing EVO-AMPM. Figure 10(a,b) compares EVO-AMPM with SoTA designs against a no-prefetching baseline.

On our traces, EVO-AMPM is the best evaluated L1D prefetcher, reaching 1.349× and 1.356× speedup on one and four cores, respectively, corresponding to an additional 4.9% and 6.7% speedup over MLOP and 7.8% and 8.4% over its AMPM seed. Its largest margins over MLOP appear on Ligra and heterogeneous mixes, adding 8.9% and 10.3%, respectively. Its advantage also persists under contention at four cores, even as baseline rankings change across workloads, indicating robust behavior across different access patterns.

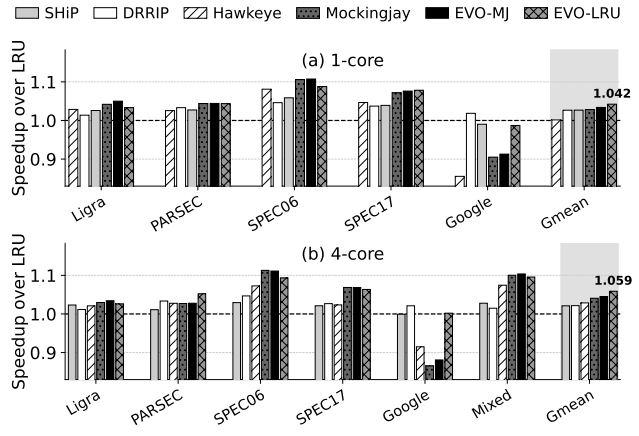


Figure 9. Geometric mean speedup over LRU by benchmark suite for (a) one core and (b) four cores. *Mixed* denotes heterogeneous four-application mixes.

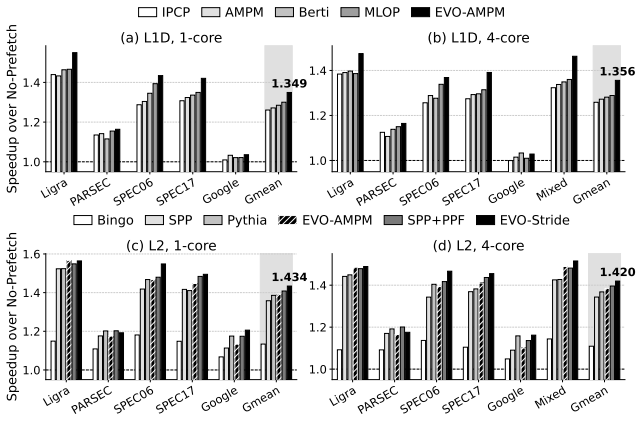


Figure 10. Geometric mean speedup over no prefetching by benchmark suite for L1D at (a) one core and (b) four cores, and L2 at (c) one core and (d) four cores. *Mixed* denotes heterogeneous four-application mixes.

EVO-AMPM's multi-engine design also remains well targeted. At four cores, it generates an average of 1.37B useful prefetches versus MLOP's 1.23B, with 40.7% accuracy versus 35.9% and 90.15% L1D load-hit rate versus 87.53%. Its shared state increases coverage, while the global issuer and adaptive feedback reject redundant requests and control aggressiveness, increasing coverage without sacrificing precision.

6.3 Evolution from a Weaker Seed

We next ask whether Agentic Architect can discover competitive designs when starting from a weaker seed.

6.3.1 Cache Replacement: Evolving from LRU. We seed evolution with LRU, producing EVO-LRU, and allow an area budget 30% above Mockingjay's to encourage exploration. Because initial searches repeatedly converged to Mockingjay's ETR, we explicitly prohibit ETR in the prompt.

Figure 9 shows EVO-LRU's geometric mean speedup alongside EVO-MJ. EVO-LRU achieves the highest geometric mean, reaching 1.042× on one core and 1.059× on four, improving on Mockingjay by approximately 1.4% and 1.7%. Its advantage comes from better balance across workload sources. On Google workloads, Mockingjay degrades by 9.5% and 13.5% on one and four cores, while EVO-LRU limits the one-core loss to 1.3% and improves four-core performance by 0.2%, recovering 8.2% and 13.7% respectively. Excluding the Google traces, EVO-LRU reaches 1.057× and 1.069×, versus Mockingjay's 1.064× and 1.073×, trading a small amount of peak performance for better balance, yielding a robust design.

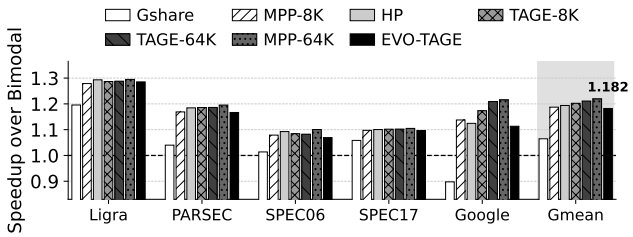


Figure 11. Geometric mean speedup over bimodal by benchmark suite.

6.3.2 Data Prefetching: Evolving from PC-Stride. We also evolve from PC-Stride at L2, allowing an area budget 30% above Pythia's. The search produces EVO-Stride, evaluated against SoTA L2 prefetchers and EVO-AMPM placed at L2.

Figure 10(c,d) shows that EVO-Stride is the strongest L2 design overall, reaching 1.434× and 1.420× speedup on one and four cores. It exceeds SPP+PPF, the strongest published baseline overall, by 2.6% and 2.0%, respectively. Its largest gains appear on SPEC06, where it leads SPP+PPF by 6.8% on one core and 5.0% on four. On the unseen Google traces, it also improves over SPP+PPF by 3.2% and 2.6%.

EVO-AMPM also transfers well across cache levels, ranking third overall despite being evolved only at L1D, without exposure to L2 access streams, latencies, or bandwidth conditions. Together with EVO-LRU, these results highlight a seed tradeoff: mature seeds provide a robust foundation and favor refinement, whereas simpler seeds leave more room for broader exploration and can produce competitive designs.

6.4 Branch Prediction

We seed evolution with TAGE-SC-L and optimize cost efficiency rather than raw speedup, producing EVO-TAGE. Figure 11 illustrates the predictors' geometric mean IPC across the five studied suites while, Figure 12 plots geometric mean IPC against modeled predictor area and power, normalized to Bimodal. EVO-TAGE retains 97.62% and 96.98% geometric

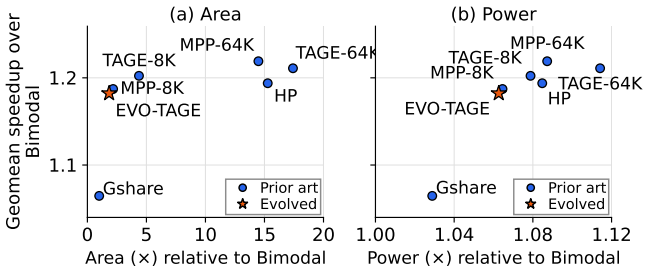


Figure 12. Geometric mean IPC speedup vs. (a) CACTI-modeled predictor area, (b) CACTI+McPAT modeled processor power, normalized to Bimodal's corresponding values.

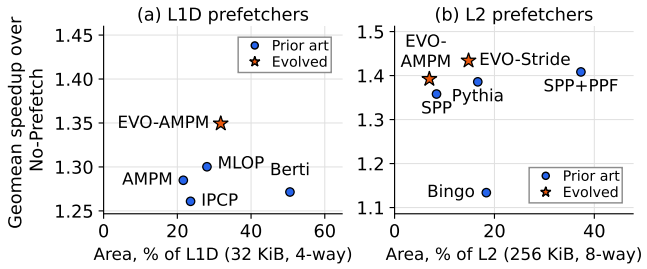


Figure 13. Single-core IPC speedup vs. CACTI-modeled chip area, normalized to the area of the cache served, for (a) L1D and (b) L2 prefetchers.

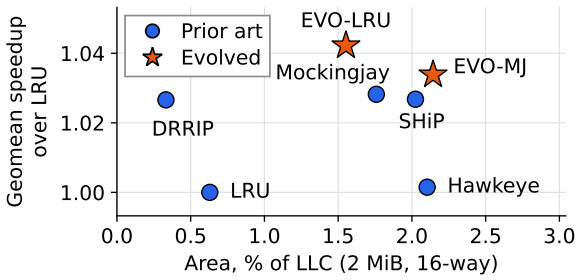


Figure 14. Single-core IPC speedup vs. CACTI-modeled chip area, normalized to LLC area, for the replacement designs.

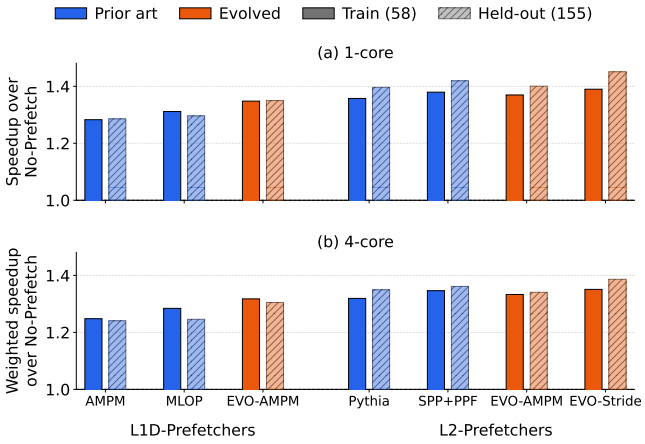


Figure 15. (a) Single-core geometric mean IPC speedup and (b) Four-core geometric mean weighted IPC speedup on 58 training traces vs. 155 held-out traces, for L1D-placed and L2-placed prefetchers.

mean IPC of its seed and the best-performing 64 KiB MPP respectively, while taking up only 10.58% and 12.72% of their area and reducing processor power by 4.62% and 2.26%. Except for Ligra, its performance does not match that of its closest area-wise mechanism, the 8 KiB MPP variant, trailing by 0.43%. Still, it occupies 15.48% less area and consumes 0.18% less power across all workloads. Despite ranking sixth in terms of performance, it remains on the IPC vs. Area and IPC vs. Power Pareto frontiers, and achieves the evolutionary objective by being 9.228× more area efficient than its seed (IPC/mm^2) and 1.024× more power efficient (IPC/W).

Overall, although seeded with TAGE-SC-L, the framework was able to recognize the area inefficiency of using large tagged tables and steer its evolved architectural policy towards a simpler perceptron design. Despite its small state size and lack of complex signals compared to MPP, it was able to achieve comparable performance across all workload suites at a fraction of the SoTA mechanism's area.

6.5 Area Consumption of the Evolved Designs

Next, we evaluate the hardware cost of the evolved mechanisms. Figures 13 and 14 show single-core geometric mean speedup against CACTI-modeled area, expressed as a percentage of the cache each design serves.

At L1D, EVO-AMPM is the fastest design while occupying 31.8% of area, only 3.7% more than MLOP and 18.8% less than Berti. At L2, EVO-Stride is the fastest design while being 1.8% and 22.5% smaller than Pythia and SPP+PPF, respectively. EVO-AMPM offers a favorable tradeoff: SPP+PPF outperforms it by 1.6% while occupying 30.4% more L2 area.

Cache replacement tells a similar story, with each evaluated design occupying less than 3% of LLC area. EVO-MJ consumes 2.1%, 0.4% more than Mockingjay, while EVO-LRU consumes 1.6% and achieves the highest overall speedup. Across both use cases, Agentic Architect produces competitive designs respecting their hardware constraints.

6.6 Generalization to unseen workloads

Because evolution is rewarded on 58 or 64 training traces, depending on the use case, we test if its gains generalize to 155 and 127 held-out traces, including the unseen Google workloads. Figures 15, 16, and 17 report geometric mean speedup for prefetching, replacement and branch prediction respectively. For prefetching and replacement the four-core geometric mean weighted speedup is also reported.

Prefetcher rankings generalize well: EVO-AMPM remains the strongest L1D design and EVO-Stride the strongest L2 design on held-out traces, both for one and four cores. Their

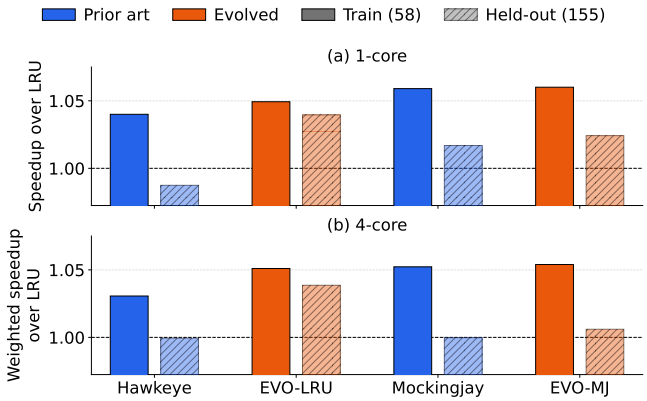


Figure 16. (a) Single-core geometric mean IPC speedup and (b) Four-core geometric mean weighted IPC speedup on 58 training vs. 155 held-out traces, for replacement designs.

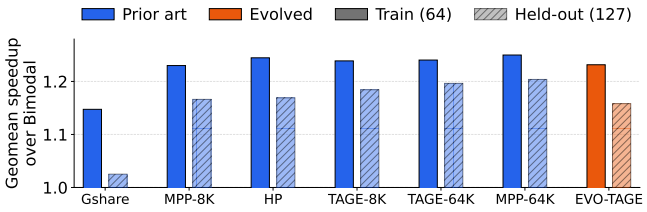


Figure 17. Geometric mean IPC speedup on 64 training traces vs. 127 held-out traces, for branch predictors.

margins over the strongest prior-art designs widen rather than shrink.

Replacement designs also generalize well, with one notable shift. On training traces, EVO-MJ reaches 1.060× at one core, essentially matching Mockingjay's 1.059×, while EVO-LRU reaches 1.049×. On held-out traces, EVO-LRU is best at 1.040×, ahead of EVO-MJ (1.024×) and Mockingjay (1.017×), consistent with its better balance on the unseen Google traces. Similarly, at four cores, EVO-LRU remains best on held-out traces at 1.039×, while EVO-MJ slightly exceeds Mockingjay (1.006× vs. 1.000×).

The same generalization trend holds for branch prediction. The only exception is EVO-TAGE outperforming the 8 KiB variant of MPP on the training set by a marginal 0.12% but being 0.7% slower on the larger hold-out set.

The gains also persist over longer simulations. Evolution uses 50M warmup and 200M measured instructions per trace, whereas final evaluation uses 200M warmup and 1B measured instructions. This also motivated excluding traces whose short-window behavior could bias the search. For example, we drop `607.cactuBSSN_s` (SPEC17), whose replacement sensitivity is specific to the short evolution window and disappears in the longer final evaluation. This emphasizes the importance of a well structured training set including diverse and well behaved traces.

7 Related Work

LLM-driven evolutionary search. ELM [31] first used LLMs as mutation operators within an evolutionary loop, and FunSearch [42] showed that this approach can produce mathematical advances. AlphaEvolve [39] extended the paradigm to general code optimization, motivating a family of open-source frameworks [6, 14, 20, 30, 33, 49, 55]. These frameworks explore different strategies for sample efficiency, mutation diversity, and search adaptation. We build on OpenEvolve [49] and apply this to CPU microarchitecture, where effective search requires hardware budgets, representative training workloads, and support for different architectural domains, objectives, and evaluation tools.

Machine Learning for systems. Prior work applies learning and automated search to index structures [29], query optimization [35], and program autotuning [4]. ADRS [15] uses LLM-driven evolution to discover algorithms for cloud scheduling, LLM inference, and transaction management. Concurrent work ArchAgent [19] explores cache replacement through AlphaEvolve, while ArchAgent v2 [17] provides a framework for evolving only multilevel prefetchers using staged search and hardware-size feedback. Sankaralingam [43] proposes an automated "Idea Factory" for architecture research. Our work is the first to propose an end-to-end, domain-independent, open-source framework that makes the seed, optimization objective, hardware budget, and training workload explicit architect-controlled dimensions of the search. We study these choices across cache replacement, prefetching, and branch prediction, including their effects on search behavior, hardware feasibility, and generalization to unseen workloads, and examine the designs and mechanisms toward which evolution converges.

8 Conclusion

We introduce Agentic Architect, an agentic CPU microarchitecture optimization framework combining LLM-driven evolution with cycle-accurate simulation and hardware cost modeling. Across prefetching, cache replacement, and branch prediction, Agentic Architect matches or improves prior art. Its evolved L1D and L2 prefetchers exceed MLOP by an additional 4.9%/6.7% and SPP+PPF by 2.6%/2.0% on one/four cores. The best evolved replacement design improves Mockingjay by 1.4%/1.7%, while the branch predictor retains 97.62% of TAGE-SC-L's geometric mean IPC at 10.58% of its area.

Our results support agentic co-design, where LLM-driven search accelerates exploration within architect-defined constraints. Seed choice shapes the search trajectory, objectives and hardware budgets define tradeoffs, and training workloads influence generalization. Agentic Architect makes these choices explicit and configurable, enabling practical agentic exploration of microarchitectural designs. The framework will be open-sourced upon publication.

References

[1] 2017. The 2nd Cache Replacement Championship (CRC2). https://crc2.ece.tamu.edu/. Held in conjunction with ISCA 2017.

[2] 2025. Google Workload Traces Version 2. https://console.cloud.google.com/storage/browser/external-traces-v2. Accessed: 2026-09-09.

[3] 2026. The 4th Data Prefetching Championship (DPC4). https://sites.google.com/view/dpc4-2026. Held in conjunction with HPCA 2026. Artifacts: https://github.com/CMU-SAFARI/DPC4.

[4] Jason Ansel, Shoaib Kamil, Kalyan Veeramachaneni, Jonathan Ragan-Kelley, Jeffrey Bosboom, Una-May O'Reilly, and Saman Amarasinghe. 2014. OpenTuner: An Extensible Framework for Program Autotuning. In *Proceedings of the 23rd International Conference on Parallel Architectures and Compilation (PACT)*. 303–316.

[5] Anthropic. 2026. *Claude Opus 5 System Card*. Technical Report. Anthropic. https://www.anthropic.com/claude-opus-5-system-card

[6] Henrique Assumpç ao, Diego Ferreira, Leandro Campos, and Fabricio Murai. 2025. CodeEvolve: an Open Source Evolutionary Coding Agent for Algorithmic Discovery and Optimization. *arXiv preprint arXiv:2510.14150* (2025).

[7] Jean-Loup Baer and Tien-Fu Chen. 1991. An effective on-chip preloading scheme to reduce data access penalty. In *Proceedings of the 1991 ACM/IEEE Conference on Supercomputing* (Albuquerque, New Mexico, USA) *(Supercomputing '91)*. Association for Computing Machinery, New York, NY, USA, 176–186. doi:10.1145/125826.125932

[8] Mohammad Bakhshalipour, Mehran Shakerinava, Pejman Lotfi-Kamran, and Hamid Sarbazi-Azad. 2019. Bingo Spatial Data Prefetcher. In *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 399–411. doi:10.1109/HPCA.2019.00053

[9] Rajeev Balasubramonian, Andrew B Kahng, Naveen Muralimanohar, Ali Shafiee, and Vaishnav Srinivas. 2017. CACTI 7: New tools for interconnect exploration in innovative off-chip memories. *ACM Transactions on Architecture and Code Optimization (TACO)* 14, 2 (2017), 1–25.

[10] Rahul Bera, Konstantinos Kanellopoulos, Anant V. Nori, Taha Shahroodi, Sreenivas Subramoney, and Onur Mutlu. 2021. Pythia: A Customizable Hardware Prefetching Framework Using Online Reinforcement Learning. In *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1121–1137. doi:10.1145/3466752.3480114

[11] Rahul Bera, Anant V. Nori, Onur Mutlu, and Sreenivas Subramoney. 2019. DSPatch: Dual Spatial Pattern Prefetcher. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) *(MICRO-52)*. Association for Computing Machinery, New York, NY, USA, 531–544. doi:10.1145/3352460.3358325

[12] Eshan Bhatia, Gino Chacon, Seth Pugsley, Elvira Teran, Paul V. Gratz, and Daniel A. Jiménez. 2019. Perceptron-based prefetch filtering. In *Proceedings of the 46th International Symposium on Computer Architecture* (Phoenix, Arizona) *(ISCA '19)*. Association for Computing Machinery, New York, NY, USA, 1–13. doi:10.1145/3307650.3322207

[13] Christian Bienia, Sanjeev Kumar, Jaswinder Pal Singh, and Kai Li. 2008. The PARSEC benchmark suite: characterization and architectural implications. In *Proceedings of the 17th International Conference on Parallel Architectures and Compilation Techniques* (Toronto, Ontario, Canada) *(PACT '08)*. Association for Computing Machinery, New York, NY, USA, 72–81. doi:10.1145/1454115.1454128

[14] Mert Cemri, Shubham Agrawal, Akshat Gupta, Shu Liu, Audrey Cheng, Qiuyang Mang, Ashwin Naren, Lutfi Eren Erdogan, Koushik Sen, Matei Zaharia, Alex Dimakis, and Ion Stoica. 2026. AdaEvolve: Adaptive LLM Driven Zeroth-Order Optimization. arXiv:2602.20133 [cs.NE] https://arxiv.org/abs/2602.20133

[15] Audrey Cheng, Shu Liu, Melissa Pan, Zhifei Li, Bowen Wang, Alex Krentsel, Tian Xia, Mert Cemri, Jongseok Park, Shuo Yang, Jeff Chen, Lakshya Agrawal, Aditya Desai, Jiarong Xing, Koushik Sen, Matei Zaharia, and Ion Stoica. 2025. Barbarians at the Gate: How AI is Upending Systems Research. *arXiv preprint arXiv:2510.06189* (2025).

[16] Nathan Gober, Gino Chacon, Lei Wang, Paul V. Gratz, Daniel A. Jiménez, Elvira Teran, Seth Pugsley, and Jinchun Kim. 2022. The Championship Simulator: Architectural Simulation for Education and Competition. *arXiv preprint arXiv:2210.14324* (2022).

[17] Abraham Gonzalez, Raghav Gupta, Akanksha Jain, Hanna Alam, Alexander Novikov, Po-Sen Huang, Matej Balog, Marvin Eisenberger, Sergey Shirobokov, Ngân Vũ, Hank Levy, Borivoje Nikolić, Sagar Karandikar, Martin Dixon, and Parthasarathy Ranganathan. 2026. ArchAgent v2: A Case Study with the Data Prefetching Championship. arXiv:2608.09874 [cs.AI] https://arxiv.org/abs/2608.09874

[18] Marius Grannæs, Magnus Jahre, and Lasse Natvig. 2011. Storage Efficient Hardware Prefetching using Delta-Correlating Prediction Tables. *J. Instr. Level Parallelism* 13 (2011). https://api.semanticscholar.org/CorpusID:14466900

[19] Raghav Gupta, Akanksha Jain, Abraham Gonzalez, Alexander Novikov, Po-Sen Huang, Matej Balog, Marvin Eisenberger, Sergey Shirobokov, Ngân Vũ, Martin Dixon, Borivoje Nikolić, Parthasarathy Ranganathan, and Sagar Karandikar. 2026. ArchAgent: Agentic AI-driven Computer Architecture Discovery. arXiv:2602.22425 [cs.AI] https://arxiv.org/abs/2602.22425

[20] Simon Hickinbotham, Hannah Sheridan, Herbie Bradley, Harry Fan, Theodoros Galanos, Ryan Zhou, Dennis Scott, and Joel Lehman. 2024. The OpenELM Library: Leveraging Progress in Language Models for Novel Evolutionary Algorithms. In *Genetic Programming Theory and Practice XX*. Springer.

[21] Yasuo Ishii, Mary Inaba, and Kei Hiraki. 2009. Access Map Pattern Matching for Data Cache Prefetch. In *Proceedings of the 23rd International Conference on Supercomputing (ICS)*. 499–500. doi:10.1145/1542275.1542349

[22] Akanksha Jain and Calvin Lin. 2016. Back to the Future: Leveraging Belady's Algorithm for Improved Cache Replacement. In *Proceedings of the 43rd Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 78–89.

[23] Aamer Jaleel, Kevin B. Theobald, Simon C. Steely Jr., and Joel Emer. 2010. High Performance Cache Replacement Using Re-Reference Interval Prediction (RRIP). In *Proceedings of the 37th Annual International Symposium on Computer Architecture (ISCA)*. 60–71. doi:10.1145/1815961.1815971

[24] J. Jeong and M. Dubois. 2006. Cache replacement algorithms with nonuniform miss costs. *IEEE Trans. Comput.* 55, 4 (2006), 353–365. doi:10.1109/TC.2006.50

[25] Daniel A. Jiménez. 2014. Strided Sampling Hashed Perceptron Predictor. *JILP Special Issue on the 4th Championship Branch Prediction Competition (CBP-4)* (2014).

[26] Daniel A. Jiménez. 2016. Multiperspective Perceptron Predictor. In *Proceedings of the 5th Championship Branch Prediction Competition (CBP-5)*.

[27] Daniel A. Jiménez and Calvin Lin. 2001. Dynamic Branch Prediction with Perceptrons. In *Proceedings of the 7th International Symposium on High-Performance Computer Architecture (HPCA)*.

[28] Jinchun Kim, Seth H. Pugsley, Paul V. Gratz, A. L. Narasimha Reddy, Chris Wilkerson, and Zeshan A. Chishti. 2016. Path Confidence based Lookahead Prefetching. In *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. doi:10.1109/MICRO.2016.7783763

[29] Tim Kraska, Alex Beutel, Ed H. Chi, Jeffrey Dean, and Neoklis Polyzotis. 2018. The Case for Learned Index Structures. In *Proceedings of the International Conference on Management of Data (SIGMOD)*.

[30] Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. 2025. ShinkaEvolve: Towards Open-Ended And Sample-Efficient Program Evolution. *arXiv preprint arXiv:2509.19349* (2025).

[31] Joel Lehman, Jonathan Gordon, Shawn Jain, Kamal Ndousse, Cathy Yeh, and Kenneth O. Stanley. 2022. Evolution through Large Models.

arXiv:2206.08896 [cs.NE] https://arxiv.org/abs/2206.08896

[32] Sheng Li, Jung Ho Ahn, Richard D. Strong, Jay B. Brockman, Dean M. Tullsen, and Norman P. Jouppi. 2009. McPAT: an integrated power, area, and timing modeling framework for multicore and manycore architectures. In *Proceedings of the 42nd Annual IEEE/ACM International Symposium on Microarchitecture* (New York, New York) *(MICRO 42)*. Association for Computing Machinery, New York, NY, USA, 469–480. doi:10.1145/1669112.1669172

[33] Gang Liu, Yihan Zhu, Jie Chen, and Meng Jiang. 2025. Scientific Algorithm Discovery by Augmenting AlphaEvolve with Deep Research. *arXiv preprint arXiv:2510.06056* (2025).

[34] Xiaoyang Lu, Rujia Wang, and Xian-He Sun. 2023. CARE: A Concurrency-Aware Enhanced Lightweight Cache Management Framework. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 1208–1220. doi:10.1109/HPCA56546.2023.10071125

[35] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Chi Zhang, Mohammad Alizadeh, Tim Kraska, Olga Papaemmanouil, and Nesime Tatbul. 2019. Neo: a learned query optimizer. *Proc. VLDB Endow.* 12, 11 (July 2019), 1705–1718. doi:10.14778/3342263.3342644

[36] Scott McFarling. 1993. *Combining Branch Predictors*. Technical Report WRL Technical Note TN-36. Digital Equipment Corporation, Western Research Laboratory.

[37] Pierre Michaud. 2016. Best-offset hardware prefetching. In *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 469–480. doi:10.1109/HPCA.2016.7446087

[38] Agustín Navarro-Torres, Biswabandan Panda, Jesús Alastruey-Benedé, Pablo Ibá nez, Víctor Vi nals Yúfera, and Alberto Ros. 2022. Berti: an Accurate Local-Delta Data Prefetcher. In *Proceedings of the 55th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. doi:10.1109/MICRO56248.2022.00072

[39] Alexander Novikov, Ngân V u, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. 2025. AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery. In *arXiv preprint arXiv:2506.13131*. https://arxiv.org/abs/2506.13131

[40] Samuel Pakalapati and Biswabandan Panda. 2020. Bouquet of Instruction Pointers: Instruction Pointer Classifier-based Spatial Hardware Prefetching. In *Proceedings of the 47th Annual International Symposium on Computer Architecture (ISCA)*. doi:10.1109/ISCA45697.2020.00021

[41] Moinuddin K. Qureshi, Daniel N. Lynch, Onur Mutlu, and Yale N. Patt. 2006. A Case for MLP-Aware Cache Replacement. In *Proceedings of the 33rd Annual International Symposium on Computer Architecture (ISCA '06)*. IEEE Computer Society, USA, 167–178. doi:10.1109/ISCA.2006.5

[42] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. 2024. Mathematical Discoveries from Program Search with Large Language Models. *Nature* 625 (2024), 468–475. doi:10.1038/s41586-023-06924-6

[43] Karthikeyan Sankaralingam. 2026. Computer Architecture's AlphaZero Moment: Automated Discovery in an Encircled World. *arXiv preprint arXiv:2604.03312* (2026).

[44] Andre Seznec. 2004. The o-gehl branch predictor. *The 1st JILP Championship Branch Prediction Competition (CBP-1)* (2004).

[45] André Seznec. 2011. A New Case for the TAGE Branch Predictor. In *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. ACM.

[46] André Seznec. 2016. TAGE-SC-L Branch Predictors Again. https://api.semanticscholar.org/CorpusID:113795057

[47] Ishan Shah, Akanksha Jain, and Calvin Lin. 2022. Effective Mimicry of Belady's MIN Policy. In *Proceedings of the 28th IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. doi:10.1109/HPCA53966.2022.00048

[48] Mehran Shakerinava, Mohammad Bakhshalipour, Pejman Lotfi-Kamran, and Hamid Sarbazi-Azad. 2019. Multi-Lookahead Offset Prefetching. https://api.semanticscholar.org/CorpusID:199570386

[49] Asankhaya Sharma. 2025. *OpenEvolve: an open-source evolutionary coding agent*. https://github.com/algorithmicsuperintelligence/openevolve

[50] Manjunath Shevgoor, Sahil Koladiya, Rajeev Balasubramonian, Chris Wilkerson, Seth H. Pugsley, and Zeshan Chishti. 2015. Efficiently prefetching complex address patterns. In *Proceedings of the 48th International Symposium on Microarchitecture* (Waikiki, Hawaii) *(MICRO-48)*. Association for Computing Machinery, New York, NY, USA, 141–152. doi:10.1145/2830772.2830793

[51] Julian Shun and Guy E. Blelloch. 2013. Ligra: a lightweight graph processing framework for shared memory. In *Proceedings of the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (Shenzhen, China) *(PPoPP '13)*. Association for Computing Machinery, New York, NY, USA, 135–146. doi:10.1145/2442516.2442530

[52] Stephen Somogyi, Thomas F. Wenisch, Anastassia Ailamaki, Babak Falsafi, and Andreas Moshovos. 2006. Spatial Memory Streaming. In *Proceedings of the 33rd Annual International Symposium on Computer Architecture (ISCA)*. 252–263. doi:10.1109/ISCA.2006.38

[53] Standard Performance Evaluation Corporation. 2006. SPEC CPU2006 Benchmark Suite. https://www.spec.org/cpu2006/

[54] Standard Performance Evaluation Corporation. 2017. SPEC CPU2017 Benchmark Suite. https://www.spec.org/cpu2017/

[55] Yiping Wang, Shao-Rong Su, Zhiyuan Zeng, Eva Xu, Liliang Ren, Xinyu Yang, Zeyi Huang, Xuehai He, Luyao Ma, Baolin Peng, Hao Cheng, Pengcheng He, Weizhu Chen, Shuohang Wang, Simon Shaolei Du, and Yelong Shen. 2025. ThetaEvolve: Test-time Learning on Open Problems. *arXiv preprint arXiv:2511.23473* (2025).

[56] Carole-Jean Wu, Aamer Jaleel, Will Hasenplaugh, Margaret Martonosi, Simon C. Steely, and Joel Emer. 2011. SHiP: Signature-based Hit Predictor for High Performance Caching. In *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 430–441. doi:10.1145/2155620.2155671

[57] Vinson Young, Aamer Jaleel, and Moin Qureshi. 2017. SHiP++: Enhancing Signature-Based Hit Predictor for Improved Cache Performance. In *2nd Cache Replacement Championship (CRC), held in conjunction with ISCA*.

[58] Siavash Zangeneh, Stephen Pruett, Sangkug Lym, and Yale N. Patt. 2020. BranchNet: A Convolutional Neural Network to Predict Hard-To-Predict Branches. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 118–130. doi:10.1109/MICRO50266.2020.00022